

Embedding 的深度学习本质

Token ID 怎样变成一个可学习向量 · 从查表到 Gradient · 重新连接 LLM 输入层

本讲定位：你以前在 LLM 课程里已经见过 Token Embedding。这一讲不再把它当成“把 Token 变成向量”的一句定义，而是从参数矩阵、Shape、查表、One-Hot 等价、Backward、位置与上下文表示重新理解它。

学完后你应该能够：

看到 `nn.Embedding(V, D)` 时，解释 `V`、`D`、`weight.shape` 和参数量。

手算一个 Token ID 如何从 Embedding Matrix 中查出一行向量。

解释为什么 `lookup` 与 `one-hot @ E` 数学等价，但工程上通常不显式构造 `one-hot`。

判断 (B, T) Token IDs 为什么会变成 (B, T, D) Embeddings。

解释 Backward 时为什么只给被访问的词表行产生梯度，以及重复 Token 为什么累加梯度。

区分 Token Embedding、位置机制与 Transformer 后的 Contextual Hidden State。

把这些基础重新接回 LLM / Transformer / Agent Systems。

1. 先解决根问题：Token ID 为什么不能直接当“数值特征”？

Tokenizer 把文本切成 Token 后，会给每个 Token 一个整数 ID。例如词表里可能有 Token 1532、1533、8001。这个整数首先只是索引编号，不是带有大小意义的连续数值。

如果把 Token ID 直接当成一个标量送进神经网络，就会暗含一种很危险的假设：ID=8000 比 ID=8 “大 1000 倍”，或者 ID=1533 与 ID=1532 在语义上一定更接近。实际上，词表编号通常不具备这种数学意义。

Embedding 要解决什么问题？

我们需要给每个离散 Token 一个可以参与神经网络矩阵计算的连续向量，而且这些向量不是人手规定，而应该通过训练自动学习。于是引出 Embedding：

Embedding = 给每个离散 ID 分配一个可学习向量。
输入是整数索引；输出是一个 D 维浮点向量。

输入、输出、内部发生什么？

对象	Shape / 类型	含义
Token ID	标量整数，例如 3	告诉模型查词表的第几行
Embedding Matrix E	(V,D)	V 行代表 V 个 Token；每行 D 个可训练参数
输出向量	(D,)	被选中 Token 当前的向量表示

所以 Embedding 并不是先做复杂神经网络计算。它最基本的动作非常朴素：按 ID 查矩阵的一行。复杂性来自这张表中的数字会在训练中不断被 Gradient 更新。

2. Embedding Matrix: 本质上是一张可训练参数表

假设词表大小 $V=5$, Embedding Dimension $D=3$ 。我们定义:

$$E.shape = (V,D) = (5,3)$$

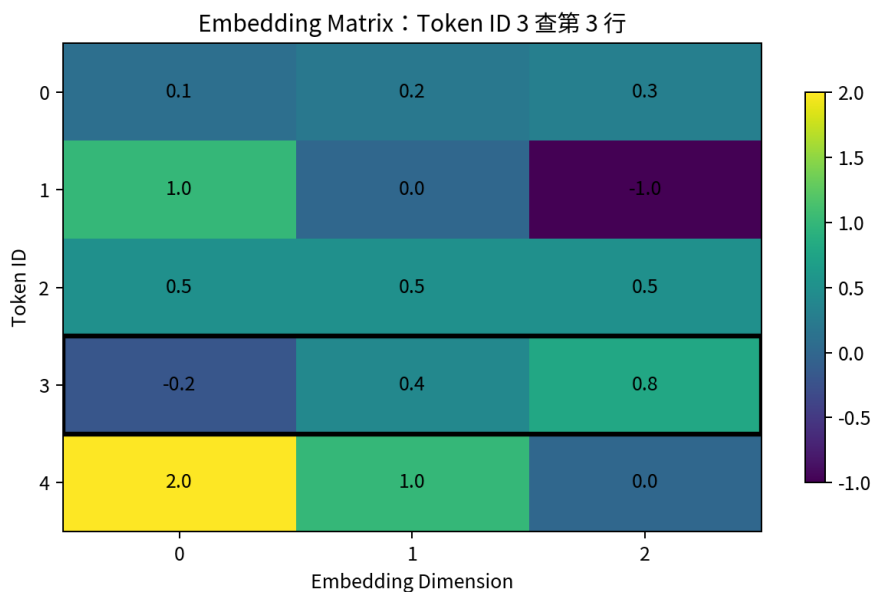


图: 5 个 Token, 每个 Token 对应 3 个可训练数值。Token ID=3 时直接查第 3 行。

具体数字例子

若 Token ID=3, 那么输出不是“数字 3”, 而是:

$$E[3] = [-0.2, 0.4, 0.8]$$

这 3 个数一开始可能只是随机初始化; 经过大规模训练后, 它们会因为下游 Loss 的 Gradient 不断变化。它们不是人工定义的“语义坐标”, 而是整个模型为了降低训练目标而学出的内部参数。

重要: Embedding 的每一维通常没有简单、稳定、可人工命名的语义。不能机械地说“第 17 维就是情感, 第 300 维就是动物”。

3. 手算：查表为什么等价于 One-Hot $\times$ Embedding Matrix?

Token ID=3 可以写成长度 $V=5$ 的 one-hot 向量：

$$\text{one_hot}(3) = [0, 0, 0, 1, 0]$$

再乘 Embedding Matrix E ：

$$[0, 0, 0, 1, 0] @ E = E[3]$$

原因很简单：矩阵乘法会把第 0、1、2、4 行乘以 0，只把第 3 行乘以 1，所以结果恰好就是第 3 行。

为什么工程上通常不真的构造 One-Hot?

如果词表 $V=50,000$ ，一个 Token 的 one-hot 就有 50,000 个位置，其中 49,999 个都是 0。显式创建这个向量再做矩阵乘法非常浪费。直接用整数 ID 做索引查表，可以得到同样结果，却避免巨大稀疏向量。

```
token_id = 3
lookup = E[token_id]          # 查表
one_hot = F.one_hot(torch.tensor(3), num_classes=V).float()
via_one_hot = one_hot @ E     # 矩阵乘法
```

实验 1 已验证：直接 lookup 与 one-hot @ E 的最大绝对误差为 0。

这里和你以前学过的 Matrix Multiplication 怎么连接？

Embedding lookup 可以用 one-hot matrix multiplication 来解释，因此它并没有脱离前面学过的矩阵知识；只是实现上利用“输入是离散索引”这个特殊结构，直接查行更加高效。

4. Shape：为什么 (B,T) 会变成 (B,T,D)?

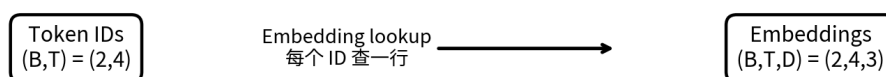
LLM 输入通常不是一个 Token，而是一批序列。假设：

B = Batch Size: 一次处理几条序列。

T = Sequence Length: 每条序列有几个 Token。

D = Embedding Dimension / Hidden Size: 每个 Token 用多少个浮点数表示。

Embedding 的 Shape 变化



如果：

```
token_ids.shape = (B,T)
```

那么每个整数 ID 都会查出一个长度 D 的向量，于是：

```
embedding(token_ids).shape = (B,T,D)
```

数字例子

实验中 B=2、T=4、D=3:

```
token_ids.shape = (2, 4)
embeddings.shape = (2, 4, 3)
```

例如 `token_ids[0,1]=3`，那么 `embeddings[0,1,:]` 就等于 Embedding Matrix 的第 3 行。

以后看到 LLM hidden state 的 (B,T,D)，第一层 D 往往就是从 Token Embedding 出来的。后续 Transformer Block 虽然不断修改数值，但通常保持这个三维基本骨架。

5. 参数量与内存：Embedding 也可能是一块很大的 Parameter

Embedding 的参数量非常直接：

$$\text{Parameter Count} = V \times D$$

例如一个示意性词表：

Vocab V	Hidden D	参数量	仅权重 fp16/bf16	仅权重 fp32
50,000	4,096	204,800,000	约 0.381 GiB	约 0.763 GiB

这只是输入 Embedding Matrix 本身。训练时如果它参与优化，还可能涉及 Gradient 与 Optimizer State；完整训练显存不能只按权重大小估算。

词表越大，发生什么？

参数量线性增加：V 翻倍，Embedding 参数量近似翻倍。

每次 lookup 仍然只查当前 Token 对应的若干行；参数总量大，不等于每次都做 $V \times D$ 的 dense matrix multiplication。

如果输入 Embedding 与输出 LM Head 做 weight tying，可以让两处共享同一张或转置关系的参数矩阵，从而影响参数量与建模结构。

科研阅读提示

看到论文里的 vocab size、hidden size、embedding tying / untied embeddings 时，不要把它们当作 tokenizer 的边角设置。它们会影响参数量、输出层结构、显存和训练行为。

6. Embedding 和 Linear 都有 Weight Matrix，但它们不是同一种操作

比较项	Embedding	Linear
典型输入	整数 ID	浮点向量
核心操作	按 ID 查行	矩阵乘法 + Bias
Weight Shape	(V,D)	PyTorch 中 (D_out,D_in)
一次 Forward 使用哪些权重	只读取被索引的行	通常所有权重都参与矩阵乘法
输出 Shape	在输入 ID Shape 后追加 D	最后一个特征维变成 D_out

两者都包含 Parameter Matrix，但 Embedding 利用了“输入是离散索引”的结构。你可以用 one-hot @ E 把它转写成特殊 Linear，但日常代码里最好保持两者概念分离。

为什么不能对 Token ID 用普通 Linear(1,D)?

如果把 ID=1532 直接作为实数输入 Linear，模型会把它当成具有连续大小关系的数值。Embedding 则把 1532 解释为“去第 1532 行取参数”，从而保留离散类别的本质。

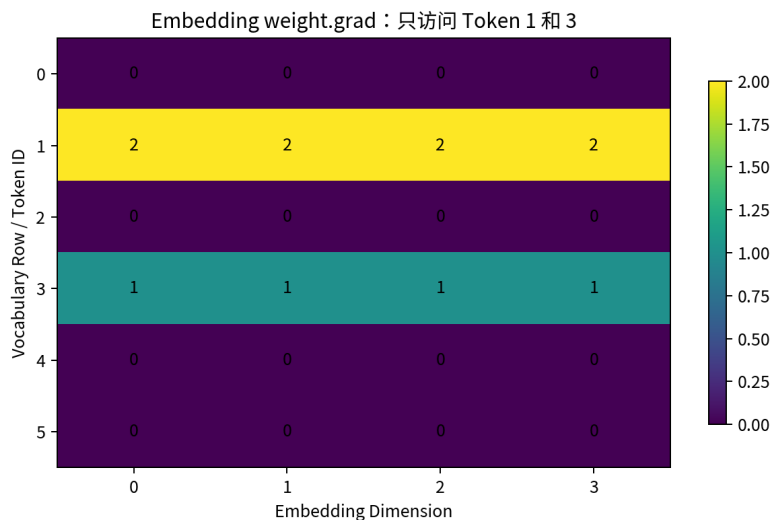
与 Q/K/V Projection 的区别

Token Embedding 把 离散 ID $\rightarrow$ D 维向量；而 Transformer 后面的 Q/K/V Projection 是把已经存在的连续 hidden vector 再做 Linear Transformation。

所以链条应该读成: Token ID $\rightarrow$ Embedding lookup $\rightarrow$ continuous hidden vector $\rightarrow$ Q/K/V Linear projections。

7. Backward: Embedding Matrix 到底怎样学？

Embedding Matrix 是普通 Parameter，因此 Loss 通过后续网络反向传播回来时，会产生对 Embedding Weight 的梯度。关键是：一次样本只访问词表中的一小部分 Token，所以 Gradient 在“词表行”这个维度上具有明显的选择性。



实验：Token 1 出现两次、Token 3 出现一次。只有第 1、3 行收到非零梯度。

重复 Token 为什么会累加？

实验输入 IDs = [1,3,1]，Loss 是所有输出元素求和。每访问一次某行，就给该行贡献一个全 1 梯度，因此：

$$\text{grad}[\text{row } 1] = [2, 2, 2, 2]$$

$$\text{grad}[\text{row } 3] = [1, 1, 1, 1]$$

这和第 23 讲讲过的“同一变量被多条路径使用时，Gradient 在汇合处相加”完全一致。Embedding 没有新数学，只是路径汇合发生在同一个 vocabulary row 上。

科研含义：一个 Token 在 Batch / Sequence 中出现多少次、出现在什么上下文，会共同影响这行 Embedding Parameter 的梯度。

8. Padding、Special Tokens 与 padding_idx

序列 Batch 往往需要把不同长度样本补齐到同一 T。补出来的位置通常使用 Padding Token。Padding 只是对齐容器长度，很多任务并不希望它像正常 Token 一样被学习。

```
embedding = nn.Embedding(
    num_embeddings=V,
    embedding_dim=D,
    padding_idx=0
)
```

实验中 IDs 为 [0,2,0,4]，其中 0 是 padding_idx。Backward 后：

Vocabulary Row	Gradient
Padding row 0	[0,0,0]
Token row 2	[1,1,1]
Token row 4	[1,1,1]

这说明普通使用路径下，padding_idx 对应行不会收到 Gradient 更新。

Special Token 不是“天然理解特殊意义”

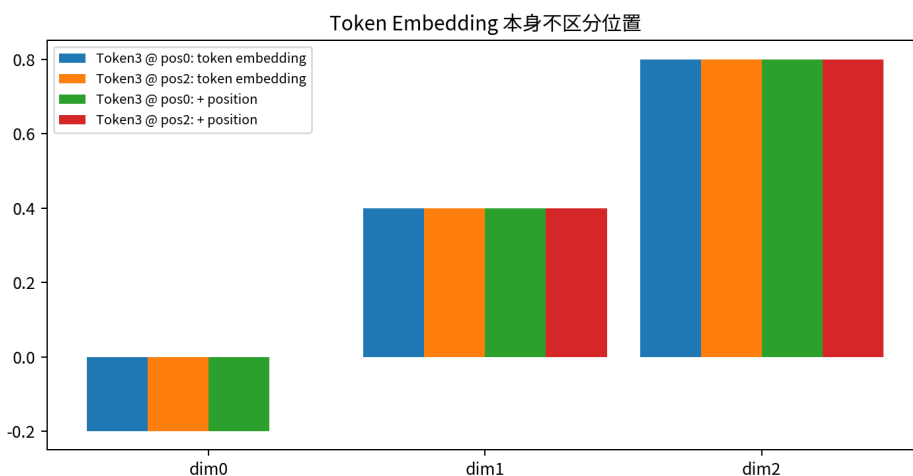
BOS、EOS、SEP、工具调用标记、Agent protocol token 等，本质上也只是词表里的离散 ID。它们之所以形成特殊功能，是因为训练数据、目标和系统结构让模型学会如何使用它们，而不是因为 Embedding API 赋予了语义。

Tokenizer 定义“有哪些 ID”；Embedding Matrix 为这些 ID 提供可训练连续向量。两者属于不同层。

9. Token Embedding 本身不知道“位置”

如果同一个 Token ID=3 在位置 0 和位置 2 出现, 纯 Token Embedding 都会查同一行 $E[3]$ 。因此:

$$\text{TokenEmbedding}(\text{token}=3, \text{pos}=0) = \text{TokenEmbedding}(\text{token}=3, \text{pos}=2)$$



实验中纯 Token Embedding 的最大差异正好为 0; 加入一个示意性 position vector 后差异变为 0.2。

那 Transformer 怎样知道顺序?

早期 Transformer / GPT 可使用 additive positional embedding: 把 Token Embedding 与 Position Embedding 相加。

很多现代 LLM 使用 RoPE 等位置机制。RoPE 通常不是简单把一个位置向量加在 Token Embedding 上, 而是在 Attention 中对 Q/K 施加位置相关旋转。

所以不要把“Embedding”一词自动等同于“Token + Position”。论文或代码必须继续看位置机制具体放在哪里。

10. Token Embedding 不等于 Contextual Embedding / Hidden State

这是初学者非常容易混淆的一点。同一个 Token 在 Embedding lookup 之后拿到的是同一行基础向量，但经过 Transformer 后，它的 hidden state 会因为周围上下文不同而不同。

两个层次

层次	同一个 Token 在不同句子里是否相同？	原因
Token Embedding lookup	通常相同	只由 Token ID 决定（暂不考虑位置机制）
Transformer 后的 contextual hidden state	通常不同	Self-Attention、MLP、Residual、Norm 把上下文信息写入表示

例如“bank”在“river bank”和“bank account”里，初始 Token Embedding 可以相同，但经过若干 Transformer Block 后，Contextual Representation 会因为上下文不同而分化。

Embedding 是起点，不是最终语义。后续网络会把上下文、位置和任务相关信息逐层写入 hidden state。

与 Agent Systems 的关系

Agent 的规划、工具调用、记忆、观察等文本最终仍通过 Tokenizer -> Token IDs -> Embedding 进入 LLM。Agent 架构可以很复杂，但底座模型的输入数值化仍从这个最基础的表示层开始。

11. Output Projection 与 Weight Tying: Embedding Matrix 还可能在输出端再次出现

语言模型最后需要把 hidden state h 映射成整个词表 V 个 Token 的 logits。最直接的做法是：

$$\text{logits} = h @ W_{\text{out}} + b, \text{ logits.shape} = (\dots, V)$$

如果 hidden size D 与输入 Embedding dimension 一致，可以让输出矩阵与输入 Embedding Matrix 共享参数（常称 weight tying，具体实现依模型而异）。

位置	典型 Shape	作用
Input Embedding E	(V,D)	Token ID $\rightarrow$ D 维向量
LM Head W_{out}	(D,V) 或以转置形式使用	D 维 hidden $\rightarrow V$ 个 logits

为什么这件事值得注意？

- 影响参数量。
- 让输入端和输出端共享同一套词表几何结构。
- 阅读 checkpoint / model config 时，需要确认到底是 tied 还是 untied。

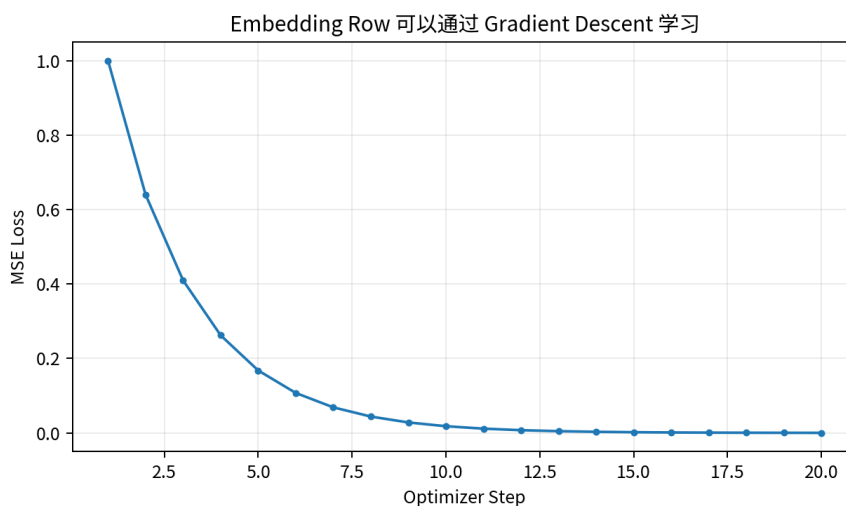
不要机械认为所有 LLM 都一定 weight tying。正确做法是看具体模型配置与实现。

12. 一个 Embedding Row 的训练：从 0 向目标向量移动

实验 6 把 Embedding Matrix 全部初始化为 0，只训练 Token 1 和 Token 3:

Token 1 的目标向量: $[1, -1]$

Token 3 的目标向量: $[-1, 1]$



20 个 SGD Step 后:

$$E[1] \approx [0.9885, -0.9885]$$

$$E[3] \approx [-0.9885, 0.9885]$$

而从未出现过的 Token 2 仍然是 $[0, 0]$ 。这个极小实验直接展示了:

Embedding 向量不是查字典得到的固定语义向量；它本身就是可训练 Parameter，Loss 会通过 Backpropagation 修改被使用的行。

真实 LLM 里会更复杂

真实预训练不会直接规定“Token 1 应该等于哪个向量”。它通常通过 next-token prediction 的 Cross Entropy 间接训练。Embedding Row 得到什么表示，是整个 Transformer、上下文和训练目标共同作用的结果。

13. PyTorch nn.Embedding：把底层原理映射到真实代码

```
import torch
import torch.nn as nn

V = 50000      # vocabulary size
D = 4096       # embedding / hidden dimension
embedding = nn.Embedding(V, D, padding_idx=0)

token_ids = torch.tensor([[12, 98, 501],
                           [12, 77, 0]], dtype=torch.long)

x = embedding(token_ids)

print(embedding.weight.shape) # (50000, 4096)
print(token_ids.shape)       # (2, 3)
print(x.shape)                # (2, 3, 4096)
```

逐项解释

num_embeddings=V: 有多少个可索引的词表行。

embedding_dim=D: 每行有多少个可训练数值。

weight: 真正的 Parameter Matrix, Shape=(V,D)。

输入 dtype: Token IDs 通常是整数 LongTensor, 因为它们是索引, 不是连续特征。

输出: 在原 ID Tensor 的 Shape 末尾追加一个 D。

常见报错怎么理解?

如果 Token ID 超出 $[0, V-1]$, 本质是“查了不存在的行”; 如果错误地把浮点 Tensor 当索引, 则类型不符合 lookup 语义。遇到这类错误, 要从索引表的底层原理定位, 而不是盲试 API。

14. 从文本到 Transformer：把以前零散的 LLM 概念重新连起来

阶段	输入 -> 输出	本课程中对应知识
Tokenizer	Text -> Token IDs	离散符号转换，不属于神经网络矩阵计算本身
Token Embedding	(B,T) -> (B,T,D)	本讲：按行 lookup，可训练 Parameter
位置机制	为表示注入顺序	Additive Position / RoPE 等
Transformer Block	(B,T,D) -> (B,T,D)	Attention + MLP + Residual + Norm
LM Head	(B,T,D) -> (B,T,V)	Linear projection 到 vocabulary logits
Cross Entropy	logits + target -> scalar loss	第18讲
Backward + AdamW	loss -> gradients -> update	第19-40讲

现在你以前 LLM 课程里见过的 Token、Embedding、Transformer、Cross Entropy、Backpropagation、AdamW 应该能重新连接成一个完整底层计算链。

下一阶段讲 Q/K/V 与 Attention 时，输入的 X 就可以理解为这种 (B,T,D) hidden representation。

15. 常见误区：这些说法哪里不严谨？

误区 1：Token ID 越接近，语义越接近

错误。ID 首先是索引编号。语义关系来自训练后的向量空间，而不是整数大小。

误区 2：Embedding 就是预训练词向量，训练时不变

错误。`nn.Embedding` 默认是可训练 `Parameter`；是否冻结取决于具体训练设置。

误区 3：Embedding 输出已经包含完整上下文语义

错误。初始 Token Embedding 只是 Transformer 的输入表示；上下文主要发生在后续层。

误区 4：同一个 Token 在句子不同位置，Embedding 自动不同

纯 Token Embedding 查同一行。位置信息需要额外机制。

误区 5：未出现在当前 Batch 的 Token 永远不会变化

对于这一 Batch 的直接 Embedding 梯度，它们可以是 0；但整个训练过程有大量 Batch，而且如果采用权重衰减、共享参数等机制，具体更新行为还要看优化器与模型结构。

误区 6：Embedding 空间里两个向量接近，就能直接断言模型“理解”它们相同

向量距离是可测现象，但“理解”的结论需要更严格的行为证据、任务定义与实验设计。

16. 科研视角：论文里出现 Embedding 时应该检查什么？

检查项	为什么重要
Vocabulary / Tokenization	改变 V、序列长度、子词切分与输入统计
Embedding Dimension / Hidden Size	影响参数量、表示容量以及后续 Transformer Shape
Trainable / Frozen	决定 Embedding Row 是否接受任务梯度
Padding / Mask	Padding 是否真正从 Loss / Attention / Gradient 中正确排除
Positional Mechanism	Token Embedding 之后位置如何进入模型
Weight Tying	影响参数量及输入/输出词表矩阵关系
Initialization	决定第0步表示尺度
Optimizer / Weight Decay	Embedding 参数是否属于 decay group，要看具体实现
Ablation	改变 Embedding 方案时是否控制其他变量与训练预算

Agent Systems 中的额外提醒

如果把工具、角色、状态、动作编码为特殊 Token，它们仍要经过同一 Embedding / Transformer 路径。研究“特殊 Token 是否有效”时，要区分 tokenizer 分配、Embedding 可训练性、训练数据覆盖和系统 Prompt 等多种因素，不要把效果全部归因于 Embedding 本身。

17. 本讲实验闭环与最终自检

本讲实验不是为了跑出“高性能模型”，而是验证 7 个底层命题：

Lookup 与 one-hot @ E 数学等价。

(B,T) Token IDs 经过 Embedding 后成为 (B,T,D)。

只有被访问的词表行收到直接梯度；同一 Token 重复出现时梯度累加。

padding_idx 的行可以不参与普通梯度更新。

Token Embedding 本身不携带位置；位置需要其他机制。

Embedding Row 可以通过 Gradient Descent 学习。

Embedding 参数量 = $V \times D$ ，规模可以很大。

5 分钟口述自检

为什么 Token ID=500 不能理解成“比 Token ID=5 大 100 倍”？

为什么 `nn.Embedding(50000,4096)` 的 `weight.shape` 是 (50000,4096)？

为什么 lookup 可以写成 one-hot @ E？为什么真实代码不这么做？

为什么输入 (B,T) 会输出 (B,T,D)？

如果 Token 17 在一个 Batch 中出现 8 次，它的 Gradient 会怎样汇合？

Token Embedding、位置机制、Contextual Hidden State 三者有什么区别？

如果这 6 个问题能够不用翻讲义完整解释，本讲就达到“必须真正掌握”的标准。

下一讲

第 53 讲将正式进入 Q、K、V：从 $X:(B,T,D)$ 出发，通过三组 Linear Projection 得到 $Q=XW_Q$ 、 $K=XW_K$ 、 $V=XW_V$ ，并逐步追踪每一个 Shape。